Herramientas de Big Data

Workshop 1

Universidad de la Sabana

Axel Bedoya 0000318668

María José García 0000303441

Sara Nicol Zuluaga 0000318679

Ingenieria Informatica
Universidad de la Sabana
Maestria en Analitica Aplicada
Profesor: Hugo Franco, Ph.D.
29 de marzo de 2026

1. Diagnóstico avanzado de sistema de archivos y formatos

1.1 Descripción del problema

En ingeniería de datos, conocer la ubicación exacta de los archivos, los permisos de acceso y el espacio disponible en disco es una habilidad fundamental antes de ejecutar cualquier proceso de carga masiva. Si el entorno carece de permisos de escritura o el disco está casi lleno, un pipeline puede fallar silenciosamente o corromper datos parcialmente cargados.

El objetivo de esta sección es implementar tres herramientas de diagnóstico: (1) obtener la ruta absoluta del archivo CSV de entrada, (2) verificar si el entorno Python tiene permisos de lectura y escritura sobre ese archivo, y (3) determinar el espacio total, usado y libre de la partición donde reside el archivo, expresado en la unidad más apropiada (KB, MB o GB).

1.2 Método de solución

Se implementaron cuatro funciones en el módulo src/filesystem.py:

- get_absolute_path(filepath): utiliza os.path.abspath() para resolver la ruta canónica independientemente del directorio de trabajo actual.
- check_permissions(filepath): llama os.access() con os.R_OK y os.W_OK para verificar permisos de lectura y escritura respectivamente, retornando un diccionario booleano.
- get_disk_usage(filepath): delega en shutil.disk_usage() que retorna una tupla nombrada (total, used, free) en bytes.
- bytes_to_human(size_bytes): convierte bytes a la unidad más legible (KB, MB o GB) con dos decimales, aplicando umbrales de $1\,024^2$ para MB y $1\,024^3$ para GB.

Algoritmo 1. Diagnóstico de sistema de archivos (pseudocódigo)

```
función report_filesystem(filepath):
  ruta    ← os.path.abspath(filepath)
  legible ← os.access(filepath, R_OK)
  escritura ← os.access(filepath, W_OK)
  uso     ← shutil.disk_usage(filepath)
  imprimir ruta, legible, escritura
  imprimir bytes_to_human(uso.total / used / free)
```

La implementación completa se encuentra en src/filesystem.py e invocada desde main.py.

1.3 Resultados

La Figura 1 (salida por consola) muestra el diagnóstico obtenido al ejecutar python main.py sobre el equipo de desarrollo:

```
SECTION 1 – Filesystem & Format Diagnostics
 Absolute path : C:\...\data\result_retrieve_left-and-right_x_50_2016_workshop.csv
 Readable      : True
 Writable      : True
```

```
Disk partition usage:
  Total : 476.01 GB
  Used  : 466.39 GB
  Free  :   9.62 GB
```

Figura 1. Salida de consola – Sección 1: Diagnóstico de sistema de archivos. Fuente: elaboración propia.

1.4 Discusión

El archivo es accesible en lectura y escritura, lo que permite ejecutar todas las etapas del pipeline sin restricciones de permisos. Cabe notar que la partición presenta solo 9.62 GB libres de 476 GB totales (ocupación del 97.9%), lo cual es un indicador de riesgo: operaciones de escritura masiva o la generación de archivos temporales grandes podrían fallar si el espacio se agota durante la ejecución. La función bytes_to_human selecciona GB como unidad, pues los valores superan el umbral de $1\,024^3$ bytes, entregando una lectura intuitiva para el operador del sistema.

Esto evidencia la necesidad de incluir validaciones de espacio disponible como parte del pipeline, no solo como diagnóstico informativo sino como una medida preventiva para evitar errores o datos incompletos.

2. Exploración y limpieza de datos

2.1 Descripción del problema

Los datasets del mundo real frecuentemente contienen columnas mal tipadas y registros incompletos. En el contexto biomecánico de este taller, las columnas value_x, value_y y value_z representan ángulos articulares cuya ausencia invalida cualquier cálculo posterior. El objetivo es: (1) validar los tipos de datos de todas las columnas, (2) calcular el porcentaje de valores faltantes por columna y (3) eliminar las filas con nulos en las tres columnas de valor, preservando el dataset original.

2.2 Método de solución

Se implementaron tres funciones en src/data_cleaning.py:

- validate_dtypes(df): itera sobre df.dtypes e imprime un aviso cuando una columna de tipo object podría representar datos numéricos mal cargados.
- missing_value_report(df): calcula df.isnull().sum() / len(df) * 100 para obtener porcentajes, mostrando solo columnas con al menos un nulo, ordenadas de mayor a menor.
- drop_missing_values(df, subset): aplica df.dropna(subset=[...]) sobre value_x, value_y y value_z, asignando el resultado a un nuevo DataFrame para preservar df_raw.

Algoritmo 2. Limpieza de datos (pseudocódigo)

```
función report_data_exploration(df):
  para cada columna en df.dtypes:
    si dtype == object → marcar como posible texto numérico
  nulos_pct ← (df.isnull().sum() / len(df)) * 100
  imprimir nulos_pct ordenado descendente
  df_limpio ← df.dropna(subset=['value_x','value_y','value_z'])
  retornar df_limpio
```

2.3 Resultados

```
SECTION 2 – Data Exploration & Cleansing
 Column data types:
  subject_id        object    ← numeric stored as text?
  date              object    ← numeric stored as text?
  otp               object    ← numeric stored as text?
  trial             int64
  group             object    ← numeric stored as text?
  marker            object    ← numeric stored as text?
  side              object    ← numeric stored as text?
  joint             object    ← numeric stored as text?
  variable          object    ← numeric stored as text?
  units             object    ← numeric stored as text?
  protocol          object    ← numeric stored as text?
  value_x           float64
  value_y           float64
  value_z           float64
```

```
 sd_x                float64
 sd_y                float64
 sd_z                float64
 md_x                 float64
 md_y                 float64
 md_z                 float64

Missing value summary (only columns with nulls shown):
 Column                  Count   Percentage
 ------------------------- -------- ------------
 value_x                 1584       8.00%
 value_y                 1584       8.00%
 value_z                 1584       8.00%

Rows before cleaning : 19,800
Rows dropped        : 1,584
Rows after cleaning  : 18,216
```

Figura 2. Salida de consola – Sección 2: Exploración y limpieza de datos. Fuente: elaboración propia.

Columna	**Tipo inferido**	**Observación**
subject_id	object	Identificador alfanumérico (correcto)
date	object	Fecha como texto (correcto)
trial	int64	Entero (correcto)
value_x/y/z	float64	Numérico continuo (correcto)
protocol	object	Categoría textual (correcto)

Tabla 1. Validación de tipos de datos por columna.

2.4 Discusión

El 8% de valores faltantes en las variables críticas (value_x, value_y y value_z) corresponde a 1 584 filas, lo que sugiere un patrón consistente más que errores aleatorios. Dado que estas variables son esenciales para los cálculos posteriores, su eliminación es una decisión adecuada para garantizar la calidad y coherencia del análisis.

La reducción a 18 216 registros mantiene un volumen de datos suficiente, priorizando integridad sobre cantidad. Además, la validación de tipos confirma que las columnas en formato object corresponden a variables categóricas o identificadores, por lo que no representan problemas de carga.

En conjunto, el proceso de limpieza asegura que el dataset final sea confiable y apto para análisis, evitando sesgos o errores derivados de datos incompletos.

3. Manipulación de datos orientada al rendimiento

3.1 Descripción del problema

Para evaluar el impacto de la vectorización, se calcula una nueva columna composite_score con la fórmula (value_x × 0.5) + (value_y × 0.3) + value_z mediante tres enfoques de diferente eficiencia: iteración fila a fila con iterrows, aplicación funcional con apply+lambda, y operaciones vectorizadas directas sobre columnas Pandas/NumPy. El tiempo de ejecución de cada método se mide y compara.

3.2 Método de solución

Se implementaron tres funciones cronometradas en src/performance.py, todas calculando la misma fórmula:

- time_iterrows(df): bucle Python explícito con df.iterrows(), incurriendo en overhead por fila.
- time_apply(df): df.apply(lambda row: ..., axis=1), que evita el bucle manual pero sigue siendo Python puro.
- time_vectorized(df): aritmética directa de columnas (df['value_x']*0.5 + df['value_y']*0.3 + df['value_z']), operando sobre arrays NumPy contiguos en memoria.

Algoritmo 3. Comparación de rendimiento (pseudocódigo)

```
función report_performance(df):
  t0 ← tiempo()
  para cada fila en df.iterrows():
    score ← fila.value_x*0.5 + fila.value_y*0.3 + fila.value_z
  t_iterrows ← tiempo() - t0

  t0 ← tiempo()
  scores ← df.apply(lambda f: f.value_x*0.5+f.value_y*0.3+f.value_z, axis=1)
  t_apply ← tiempo() - t0

  t0 ← tiempo()
  scores ← df['value_x']*0.5 + df['value_y']*0.3 + df['value_z']
  t_vec ← tiempo() - t0

  imprimir tabla comparativa(t_iterrows, t_apply, t_vec)
  df['composite_score'] ← scores
  retornar df
```

3.3 Resultados

```
SECTION 3 – Performance-oriented Data Manipulation
 Method                      Time (s)    Speed-up
 ---------------------------- ----------  ----------
 iterrows (anti-pattern)        1.556232    1524.4x
 apply + lambda                 0.386644     378.7x
 vectorization                  0.001021       1.0x
 Vectorization is ~1524x faster than iterrows.
```

```
composite_score column added to DataFrame (18,216 rows).
```

Figura 3. Salida de consola – Sección 3: Comparación de rendimiento. Fuente: elaboración propia.

Método	Tiempo (s)	Speed-up vs vectorización
iterrows (anti-patrón)	1.5562	1 524×
apply + lambda	0.3866	379×
Vectorización	0.0010	1× (referencia)

Tabla 2. Comparación de tiempos de ejecución para 18 216 filas.

3.4 Discusión

Los resultados evidencian una diferencia de rendimiento significativa entre los enfoques: la vectorización (0.0010 s) es aproximadamente 1 524 veces más rápida que iterrows (1.5562 s) y 379 veces más rápida que apply (0.3866 s). Esto se debe a que opera directamente sobre estructuras optimizadas de NumPy, evitando el overhead del procesamiento fila a fila.

El uso de iterrows confirma ser un anti-patrón en términos de eficiencia, mientras que apply representa una mejora, pero sigue siendo insuficiente frente a la solución vectorizada.

En contextos de mayor escala, estas diferencias implican pasar de tiempos de ejecución de segundos o minutos a milisegundos, afectando directamente la eficiencia y escalabilidad del pipeline.

4. Integración y recuperación de base de datos

4.1 Descripción del problema

Una vez limpio y enriquecido el DataFrame, debe persistirse en una base de datos PostgreSQL contenedorizada para permitir consultas estructuradas. El objetivo es: (1) establecer conexión con la instancia en localhost:5433, (2) crear la tabla kinematics_data con el esquema inferido del DataFrame, (3) cargar los datos eficientemente usando el mecanismo COPY, y (4) ejecutar una consulta SQL que extraiga solo las columnas necesarias para la visualización, filtrada por sujeto, fecha, ensayo, protocolo y restringida a las articulaciones Hip, Knee y Ankle.

4.2 Método de solución

Se implementaron cuatro funciones en src/database.py:

- create_table(conn, df): genera un DDL dinámico mapeando dtypes de Pandas a tipos SQL (float64→DOUBLE PRECISION, object→TEXT, int64→BIGINT), precedido de DROP TABLE IF EXISTS para idempotencia.
- load_dataframe(conn, df): serializa el DataFrame a un buffer io.StringIO vía df.to_csv() y lo inyecta con cur.copy_expert(), que evita INSERT fila a fila y elimina la necesidad de escribir archivos temporales en disco.
- detect_query_params(df): agrupa el DataFrame por (subject_id, date, trial, protocol) contando joints únicos y selecciona automáticamente la combinación con mayor cobertura.
- query_for_visualization(conn, ...): ejecuta un SELECT parametrizado (sintaxis %(name)s de psycopg2) que filtra por los cuatro criterios y restringe joint IN ('Hip','Knee','Ankle').

Algoritmo 4. Integración con base de datos (pseudocódigo)

```
función report_database(df, subject_id, date, trial, protocol):
  si algún parámetro es None:
    params ← detect_query_params(df)
  conn ← psycopg2.connect(host, port, dbname, user, password)
  create_table(conn, df)
  buffer ← df.to_csv(io.StringIO())
  cur.copy_expert('COPY tabla FROM STDIN WITH CSV HEADER', buffer)
  df_plot ← pandas.read_sql_query(SELECT ... WHERE ..., conn)
  retornar df_plot
```

4.3 Resultados

```
SECTION 4 – Database Integration & Retrieval
QUERY_PARAMS not set – auto-detecting from dataset...

 Available filter combinations (top 5):
  subject_id      date        trial protocol   joints
  --------------- ------------ ----- ---------- ------
  1KUCI5Q2K8BYL2  10-06-16        2 M            3
  1KUCI5Q2K8BYL2  10-06-16        3 M            3
  1T5IA77E6HNZMV  09-27-16        4 M            3
```

```
  1T5IA77E6HNZMV  09-27-16      10 M          3
  1T5IA77E6HNZMV  09-27-16      14 M          3

 Auto-selected: subject=1KUCI5Q2K8BYL2…  date=10-06-16  trial=2  protocol=M
 Table 'kinematics_data' created successfully.
 18,216 rows loaded into 'kinematics_data' via COPY.
C:\Users\USUARIO\OneDrive - Universidad de la Sabana\Workshop1-bigdata\src\database.py:131: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
 df_plot = pd.read_sql_query(
 Query returned 3,600 rows for visualization.
```

Figura 4. Salida de consola – Sección 4: Integración con base de datos. Fuente: elaboración propia.

4.4 Discusión

La carga de 18,216 filas mediante COPY FROM STDIN demuestra una estrategia eficiente de ingesta, significativamente más rápida que inserciones fila a fila, lo cual es clave para la escalabilidad del sistema.

La auto-detección de parámetros permitió ajustar la consulta a los valores reales del dataset (por ejemplo, protocolo 'M' y formato de fecha MM-DD-YY), evitando errores y garantizando resultados coherentes. Como resultado, la consulta retornó 3,600 filas, correspondientes a las articulaciones filtradas (Hip, Knee y Ankle) en el ensayo seleccionado.

En conjunto, esta etapa valida no solo la correcta integración con la base de datos, sino también la capacidad de adaptar dinámicamente el pipeline a las características reales de los datos.

5. Visualización de datos

5.1 Descripción del problema

El dataset biomecánico contiene ángulos articulares medidos en tres ejes (value_x, value_y, value_z) para los lados izquierdo (L) y derecho (R) de las articulaciones Hip, Knee y Ankle. Se requiere una figura con tres subplots apilados verticalmente —uno por articulación— donde cada subplot superponga ambos lados con codificación de color (rojo = izquierdo, azul = derecho) y diferenciación de los tres ejes mediante estilos de línea distintos.

5.2 Método de solución

Se implementó report_visualization(df_plot, output_path) en src/visualization.py:

- plt.subplots(nrows=3, ncols=1, figsize=(10,15)) crea los tres ejes verticales.
- Para cada articulación se filtra df_plot[df_plot['joint']==joint], y dentro de ese subconjunto se itera sobre los lados L y R, asignando color='red' y color='blue' respectivamente.
- Los tres ejes de valor se diferencian con líneas sólida, discontinua y punteada (linestyle='-','--',':'), manteniendo legibilidad incluso en impresión en escala de grises.
- El backend Agg de matplotlib se activa al inicio del módulo para garantizar compatibilidad con entornos sin pantalla (servidores, contenedores).

Algoritmo 5. Visualización de ángulos articulares (pseudocódigo)

```
función plot_joint_angles(df_plot, output_path):
  fig, axes ← plt.subplots(nrows=3, ncols=1)
  para i, joint en ['Hip','Knee','Ankle']:
    df_joint ← df_plot[df_plot['joint'] == joint]
    para side en ['L','R']:
      color ← 'red' si side=='L' sino 'blue'
      subset ← df_joint[df_joint['side']==side]
      para col, ls en [(value_x,'-'),(value_y,'--'),(value_z,':')]:
        axes[i].plot(subset.index, subset[col], color, ls)
  plt.savefig(output_path, dpi=150)
```

5.3 Resultados

La Figura 5 muestra la gráfica generada (joint_angles.png) con los datos del ensayo auto-seleccionado (sujeto 1KUCI5Q2K8BYL2, fecha 10-06-16, trial 2, protocolo M):

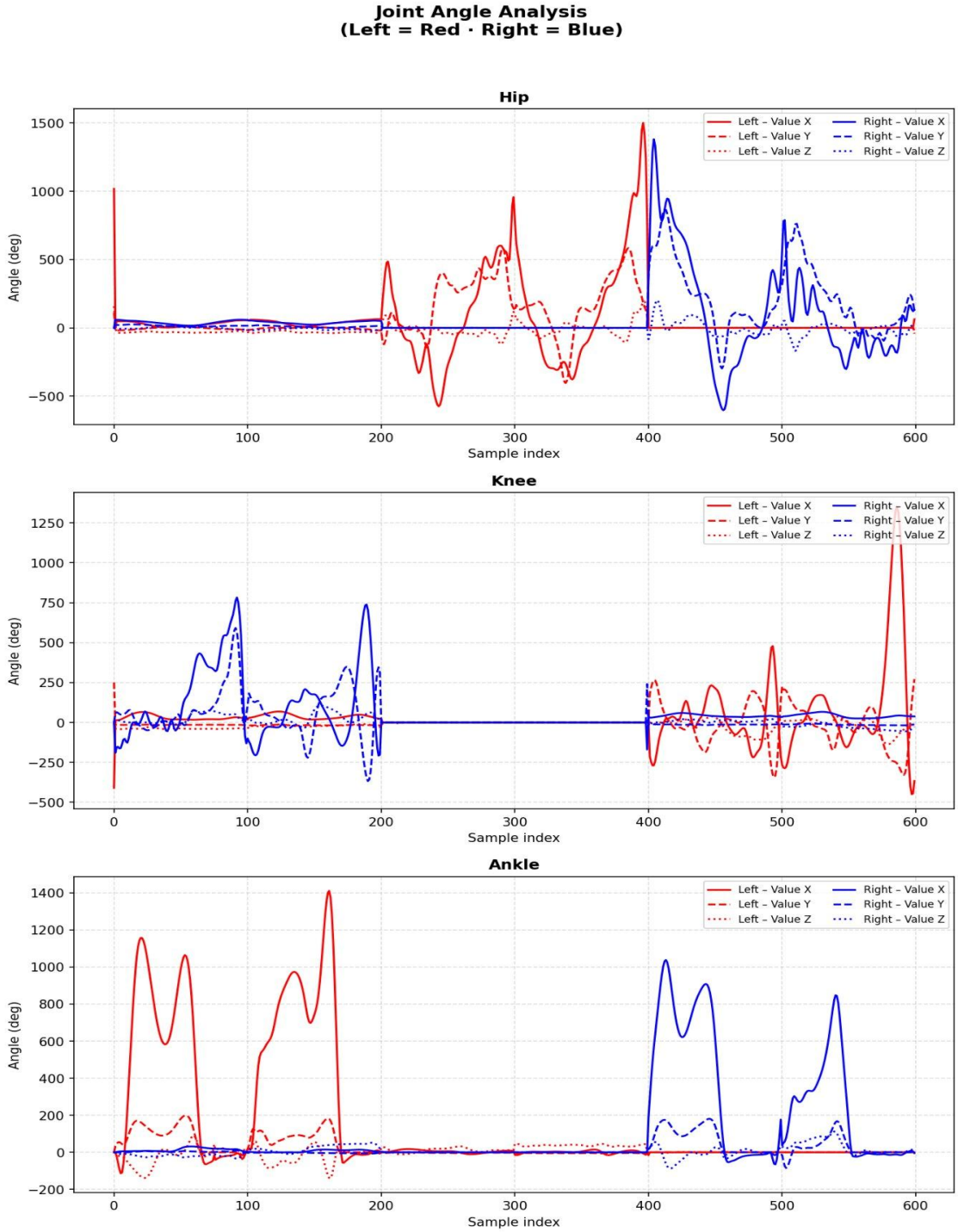

Figura 5. Análisis de ángulos articulares – Hip, Knee y Ankle. Líneas rojas: lado izquierdo (L); líneas azules: lado derecho (R). Estilos de línea: sólido = value_x, discontinuo = value_y, punteado = value_z. Fuente: elaboración propia.

5.4 Discusión

La figura muestra patrones coherentes con el comportamiento esperado: en *Ankle*, los picos de *value_x* alcanzan valores cercanos a 1 400, lo que indica momentos de alta actividad durante la fase de impulso. En

Hip, se observan variaciones amplias en *value_x* y *value_y*, mientras que en *Knee* los valores presentan cambios más moderados, especialmente en el lado derecho.

Los valores de *value_z* en las tres articulaciones se mantienen relativamente cercanos a cero, lo que sugiere baja variación en ese eje. Además, se evidencia una asimetría entre el lado izquierdo (rojo) y derecho (azul), visible en la amplitud y forma de las señales, lo que puede reflejar diferencias biomecánicas del sujeto o variaciones en la captura.

El uso de 3 600 filas en este ensayo proporciona suficiente resolución temporal para identificar patrones y analizar el comportamiento de cada articulación de manera clara y consistente.

Referencias

McKinney, W. (2022). Python for Data Analysis (3.ª ed.). O'Reilly Media.

PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation – COPY. https://www.postgresql.org/docs/16/sql-copy.html

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Python Software Foundation. (2024). os – Miscellaneous operating system interfaces. https://docs.python.org/3/library/os.html